

Binary Search Complexity

Binary Search has $O(\log n)$ time complexity because the search space is divided into half after every comparison.

1. Suppose we have 16 elements

Consider a sorted array:

```
[10, 20, 30, 40, 50, 60, 70, 80,  
90, 100, 110, 120, 130, 140, 150, 160]
```



Binary Search checks the **middle element** first.

```
16 elements  
  ↓  
 8 elements  
  ↓  
 4 elements  
  ↓  
 2 elements  
  ↓  
 1 element
```



So the number of elements becomes:

$16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

That's only **4 comparisons**.

2. Why $\log_2(n)$?

At every step, we divide n by 2.

After 1 step:

$$\frac{n}{2}$$

After 2 steps:

$$\frac{n}{2^2}$$

After 3 steps:

$$\frac{n}{2^3}$$

After **k** steps:

$$\frac{n}{2^k}$$

We stop when only **1 element** remains:

$$\frac{n}{2^k} = 1$$

Therefore:

$$n = 2^k$$

Taking logarithm base 2:

$$k = \log_2 n$$

Therefore, Binary Search requires at most:

$$O(\log_2 n)$$

In Big-O notation, the base of the logarithm doesn't matter, so we simply write:

$$O(\log n)$$

3. Example with 1,000,000 elements

This is where Binary Search becomes very powerful.

Number of elements	Approx. comparisons
10	4
100	7
1,000	10
10,000	14
100,000	17
1,000,000	20

For **1 million elements**, Binary Search needs only about **20 comparisons** in the worst case.

Because:

$$2^{20} = 1,048,576$$

So approximately 20 halvings are enough to reduce 1 million elements to 1.

4. Compare Linear Search

Linear Search:

1 → 2 → 3 → 4 → 5 → ... → n



It may need to check every element:

$$O(n)$$

Binary Search:

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$

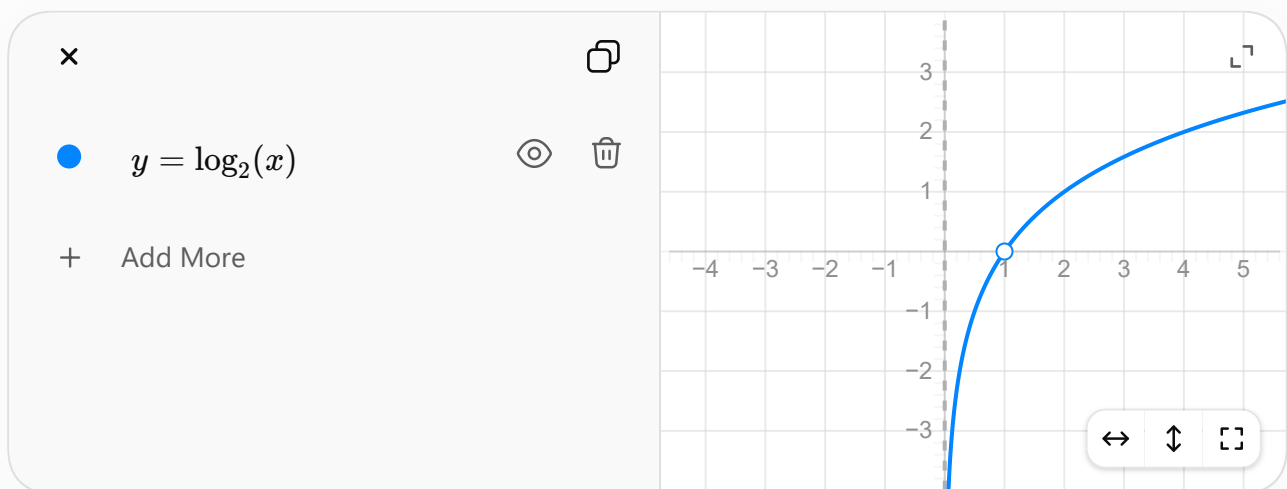


It needs:

$$O(\log n)$$

So:

Linear Search → $O(n)$
Binary Search → $O(\log n)$



Give feedback

5. Important condition

Binary Search works efficiently when the data is **sorted**.

For example:

[10, 20, 30, 40, 50, 60, 70]



If we search for 60 :

Middle = 40



```
60 > 40
```

```
→ Ignore left half
```

```
[50, 60, 70]
```

```
Middle = 60
```

```
→ Found!
```

We don't search all elements because each comparison eliminates **half of the remaining possibilities**.

Easy way to remember

Binary Search = Divide by 2 repeatedly → therefore $O(\log n)$.

The key idea is:

Number of halvings needed to reach 1 = $\log_2 n$

